



Sveučilište Josipa Jurja Strossmayera u Osijeku
Fakultet primijenjene matematike i informatike

Sveučilišni diplomski studij matematike
modul: matematika i računarstvo

Klasifikacija sportskih tekstova primjenom tehnika obrade prirodnog jezika i dubinskog učenja

Seminarski rad

Mentor: **doc.dr.sc. Domagoj Ševerdija**

Student: **Antonio Ištvanović**

Osijek, 2026

Sadržaj

1	Uvod	2
2	Skup podataka	2
3	Metodologija	3
3.1	Tokenizacija i osnovno čišćenje teksta	3
3.2	Baseline model: TF-IDF + logistička regresija	3
3.3	Neuronski model: LSTM klasifikator	3
3.3.1	Vokabular bez curenja informacija	3
3.3.2	Kodiranje i padding	4
3.3.3	Embedding i LSTM	4
3.3.4	Masked mean pooling	4
3.3.5	Klasifikacijski sloj i funkcija gubitka	4
4	Eksperimenti	5
4.1	Postavke treniranja	5
4.2	Metrika: makro F1	5
5	Rezultati	6
5.1	Treniranje i validacija	6
5.2	Testiranje	6
6	Provjera ispravnosti obrade teksta	8
7	Zaključak	9

Sažetak

U ovom radu prikazan je praktičan NLP program za klasifikaciju kratkih tekstova iz svijeta sporta. Projekt implementira dva pristupa: neuronski model dubinskog učenja temeljen na LSTM mreži u PyTorch-u i klasični baseline model temeljen na TF-IDF reprezentaciji i logističkoj regresiji. Opisani su koraci obrade teksta, izgradnje vokabulara bez curenja informacija, kodiranja i dopunjavanja (padding) sekvenci, treniranja te evaluacije pomoću makro F1 mjere. Na testnom skupu neuronski model postiže točnost 0.8929 i makro F1 0.8303. Dodatno provjeravamo ispravnost preprocessing-a kroz doctest i unittest testove.

1 Uvod

Klasifikacija teksta je jedan od najčešćih zadataka obrade prirodnog jezika: za zadani tekst potrebno je predvidjeti pripadajuću klasu (npr. tematiku, sentiment ili tip teksta). U praksi se često kao polazna točka koristi TF-IDF reprezentacija u kombinaciji s linearnim klasifikatorima jer takav pristup daje dobre rezultate uz relativno malu složenost. S druge strane, modeli dubinskog učenja (npr. RNN/LSTM) mogu učiti sekvencijske obrasce i reprezentacije iz podataka, što je korisno kada su odnosi među riječima relevantni.

U ovom seminaru prolazimo kroz cijeli pipeline od obrade teksta do treniranja i evaluacije neuronskog modela. Projekt sadrži:

- **BaselineClassifier:** TF-IDF + logistička regresija (referentna točka).
- **LSTMClassifier:** kompaktan LSTM model s embedding slojem i *masked mean pooling* agregacijom.

Naglasak je na pravilnoj pripremi podataka (posebno izgradnji vokabulara bez *data leakage*-a), odabiru prikladne metrike (makro F1) i na kraju interpretaciji dobivenih rezultata.

2 Skup podataka

Podaci se učitavaju iz `data/matches.csv` i sadrže stupce: `text` (tekstualni ulaz) i `label` (cjelobrojna oznaka klase). Iz rezultata testiranja vidimo da se klase nazivaju `PREVIEW` i `REPORT`, pomoću njih ćemo znati radi li se o najavi utakmice ili izvještaju nakon završenog susreta.

Podjela na skupove provodi se stratificirano (funkcija `stratified_split`), što je važno kada su klase neuravnotežene. Na testnom skupu se pojavljuje 22 uzorka klase `PREVIEW` i 6 uzoraka klase `REPORT`, pa korištenje stratifikacije smanjuje rizik da testni skupovi budu pristrani.

3 Metodologija

3.1 Tokenizacija i osnovno čišćenje teksta

Tekst se tokenizira jednostavnom metodom temeljenom na regularnom izrazu koji izdvaja riječi oblika:

$$\backslash\mathbf{b}\backslash\mathbf{w}+\backslash\mathbf{b}.$$

Prije izdvajanja tokena tekst se prevodi u mala slova, kako bi se smanjila varijabilnost ulaza i izbjegle razlike koje nemaju semantičku informaciju (npr. velika slova). Za tekst dobivamo sekvencu tokena:

$$t = (w_1, w_2, \dots, w_n).$$

Osim osnovne funkcije `tokenize`, implementirana je i funkcija `make_tokenizer` kao closure, a omogućuje opcionalno uklanjanje stop-riječi. U ovom projektu treniranje vokabulara i dataset koriste osnovnu tokenizaciju.

3.2 Baseline model: TF-IDF + logistička regresija

Baseline model koristi standardnu TF-IDF reprezentaciju i logističku regresiju. Za riječ w u tekstu d :

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \text{idf}(w), \quad \text{idf}(w) = \log \left(\frac{N}{1 + \text{df}(w)} \right),$$

gdje je N broj svih tekstova u skupu podataka, a $\text{df}(w)$ broj tekstova koji sadrže w . Dobiveni vektor značajki $\mathbf{x}$ koristi se kao ulaz u logističku regresiju. U binarnom slučaju:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

U implementaciji se koristi `TfidfVectorizer(max_features=5000)` i `LogisticRegression(max_iter=1000)`. Iako baseline nije centralni dio eksperimentalnog dijela ovog rada, služi kao referentna točka i osnovna provjera ispravnosti cijelog sustava.

3.3 Neuronski model: LSTM klasifikator

3.3.1 Vokabular bez curenja informacija

Za neuronski model potrebno je svaku riječ mapirati na indeks iz vokabulara. Vokabular se gradi samo na datasetu za treniranje:

$$\mathcal{V} = \{w : \text{freq}_{\text{train}}(w) \geq \text{min_freq}\},$$

gdje je u projektu `min_freq=2`. Ovaj detalj je važan jer sprječava da informacije iz skupa za treniranje utječu na vokabular (tj. na ulaznu reprezentaciju tijekom treniranja).

3.3.2 Kodiranje i padding

Nakon tokenizacije, svaki token se kodira u indeks:

$$x_i = \text{encode}(w_i) \in \{0, 1, \dots, |\mathcal{V}| - 1\}.$$

Sekvenca se reže na maksimalnu duljinu `max_len=200` i dopunjuje padding s vrijednosti `pad_idx=0`:

$$\mathbf{x} = (x_1, x_2, \dots, x_L), \quad L = 200.$$

U vokabularu su unaprijed definirani specijalni tokeni `<PAD>` i `<UNK>` s indeksima 0 i 1, a nepoznate riječi tijekom kodiranja mapiraju se na `<UNK>`.

Ovaj postupak kodiranja i paddinga implementiran je u klasi `TextDataset` (datoteka `training/dataset.py`), koja nasljeđuje PyTorchovu klasu `Dataset`. Pripremljeni tenzori zatim se grupiraju u batch pomoću `DataLoader`-a i koriste kao ulaz u neuronski model.

3.3.3 Embedding i LSTM

Indeksi riječi prolaze kroz embedding sloj:

$$\mathbf{e}_t = \mathbf{E}[x_t], \quad \mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d},$$

gdje je `embedding_dim=100`. Zatim se embedding sekvenca prosljeđuje u LSTM:

$$(\mathbf{h}_t)_{t=1}^L = \text{LSTM}((\mathbf{e}_t)_{t=1}^L),$$

pri čemu je `hidden_dim=128` i `batch_first=True`.

3.3.4 Masked mean pooling

U ovom projektu maskiranje je nužno zbog korištenja paddinga kojim se sekvence, odnosno riječi, različitih duljina dopunjuju do fiksne duljine od 200 tokena. Bez maskiranja u agregaciju bi ušli i izlazi LSTM-a koji odgovaraju padding tokenima (`<PAD>`), time bismo razrijedili reprezentaciju stvarnog teksta i smanjili utjecaj informativnih tokena.

Zbog toga se koristi maska $m_t = \mathbb{I}[x_t \neq \text{pad_idx}]$ kojom se zanemaruju vremenski koraci koji odgovaraju paddingu. Agregacija se zatim provodi kao aritmetička sredina samo nad stvarnim tokenima u sekvenci. Kako bi se izbjeglo dijeljenje s nulom u slučaju kraćih ili praznih sekvenci, duljina jedne riječi tj. sekvence se u implementaciji ograničava na najmanje 1 (`clamp(min=1)`).

3.3.5 Klasifikacijski sloj i funkcija gubitka

Pooled vektor prolazi kroz linearni sloj:

$$\mathbf{z} = \mathbf{W}\bar{\mathbf{h}} + \mathbf{b}, \quad \mathbf{z} \in \mathbb{R}^C,$$

gdje je broj klasa `num_classes=2`. Treniranje koristi **cross-entropy** gubitak:

$$\mathcal{L} = - \sum_{k=1}^C y_k \log(\hat{y}_k), \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}).$$

4 Eksperimenti

4.1 Postavke treniranja

Treniranje se izvodi u skripti `train.py` uz sljedeće postavke:

- epohe: `epochs=10`
- batch: `batch_size=16`
- optimizator: Adam (`lr=1e-3`)
- gubitak: `CrossEntropyLoss`
- seed: `seed=42`
- duljina sekvence: `max_len=200`, padding indeks: `pad_idx=0`

Podjela podataka provodi se stratificirano na omjere 80%/10%/10% (train/val/test) uz dodatne zaštite za male klase kako bi u svakom podskupu ostao dovoljan broj uzoraka po klasi.

Najbolji model se sprema prema kriteriju maksimalne vrijednosti $F1_{\text{macro}}$ mjere.

4.2 Metrika: makro F1

Osim točnosti kao glavna evaluacijska mjera koristi se $F1_{\text{macro}}$, jer bolje reflektira performanse na neuravnoteženim skupovima. Za svaku klasu:

$$F1 = \frac{2PR}{P + R}, \quad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN},$$

a makro prosjek:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c.$$

5 Rezultati

5.1 Treniranje i validacija

Ispod su prikazani rezultati dobiveni iz loga treniranja (prosječni loss i validacijski makro F1). Tijekom treniranja uočava se postupni rast validacijskog $F1_{\text{macro}}$, uz oscilacije koje su karakteristične za manji validacijski skup.

Tablica 1: Rezultati treniranja po epohama (validacija)

Epoha	Prosječni loss	$F1_{\text{macro}}$ (validacija)
1	0.6225	0.4375
2	0.5298	0.4375
3	0.5112	0.4375
4	0.4833	0.4375
5	0.4455	0.4255
6	0.3931	0.4130
7	0.2672	0.4863
8	0.3307	0.6137
9	0.2173	0.5598
10	0.2020	0.4000

Najbolji rezultat postignut je u kasnijoj fazi treniranja $F1_{\text{macro}} = 0.6137$, nakon čega dolazi do pada vrijednosti u zadnjoj epohi. Ukupno vrijeme treniranja iznosilo je 4.27 s (prema mjerenju vremena treniranja).

5.2 Testiranje

Testiranje se izvodi skriptom `evaluate.py`. Dobili smo:

$$\text{Accuracy} = 0.8929, \quad F1_{\text{macro}} = 0.8303.$$

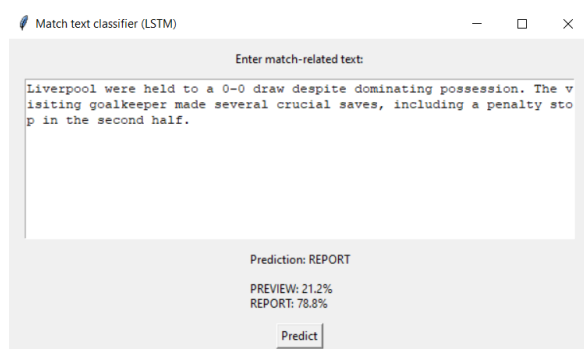
Na skupu za testiranje možemo uočiti neravnomjeran broj uzoraka po klasama (22 prema 6), pa je korišten makro F1 kako bi se performanse mjerile ravnopravno po klasama.

Detalji po klasama prikazani su u Tablici 2.

Tablica 2: Report klasifikacije na skupu za testiranje

Klasa	Precision	Recall	F1-score	Support
PREVIEW	0.91	0.95	0.93	22
REPORT	0.80	0.67	0.73	6
Accuracy	0.89 (28 uzoraka)			
Macro avg	0.86	0.81	0.83	28
Weighted avg	0.89	0.89	0.89	28

Model vrlo pouzdano klasificira tekstove iz klase **PREVIEW**, što možemo i vidjeti iz visoke vrijednosti F1 mjere (0.93). S druge strane, klasa **REPORT** pokazuje niži recall (0.67), što nam govori da dio tekstova koji su izvještaji model pogrešno svrstava u klasu **PREVIEW**. Takvo ponašanje nije čudno s obzirom na manji broj uzoraka te klase u skupu za testiranje.



Slika 1: Klasifikacija REPORT teksta



Slika 2: Klasifikacija PREVIEW teksta

Na fotografijama iznad vidimo da model s visokom sigurnosti razlikuje najave utakmica od izvještaja nakon utakmica.

6 Provjera ispravnosti obrade teksta

Budući da ispravnost obrade teksta izravno utječe na ponašanje modela, osnovne funkcije za tokenizaciju provjerene su automatskim testovima. Za provjeru očekivanog ponašanja funkcija `tokenize`, `make_tokenizer` i `batch_tokenize` koristili smo `doctest`, čime smo testirali različite primjere ulaznog teksta.

Ukupno je izvršeno sedam `doctest` testova i svi su prošli. Dodatno su implementirani `unittest` testovi za osnovne slučajeve tokenizacije, koji su također prošli. Time smo osigurali da priprema teksta daje konzistentne rezultate prije nego što se podaci koriste u treniranju modela.

7 Zaključak

U ovom radu implementiran je program za klasifikaciju sportskih tekstova, koji obuhvaća sve ključne korake NLP pipelinea, obradu teksta, treniranje i evaluaciju neuronskog modela. U projektu su dva pristupa, LSTM neuronski model koji koristi sekvencijalnu obradu teksta i jednostavni baseline model temeljen na TF-IDF reprezentaciji i logističkoj regresiji.

Rezultati pokazuju da LSTM model ima vrlo dobre performanse na skupu za testiranje (Accuracy 0.8929, $F1_{\text{macro}} = 0.8303$), pri čemu se jasno vidi razlika u ponašanju modela po klasama. Dok su tekstovi klase **PREVIEW** uglavnom točno klasificirani, klasa **REPORT** pokazala se problematičnijom, što se može pripisati manjem broju dostupnih uzoraka. Upravo iz tog razloga makro F1 mjera pokazala se prikladnijom od same točnosti za procjenu kvalitete modela.

Tijekom rada pokazali smo da ispravna priprema podataka, osobito izgradnja vokabulara, ima važan utjecaj na stabilnost i interpretaciju rezultata. Također iz oscilacija validacijskog $F1_{\text{macro}}$ vidimo osjetljivost modela na veličinu validacijskog skupa, što je očekivana posljedica ograničene količine podataka.

Moguća buduća poboljšanja su detaljnija analiza pogrešaka (npr. kroz confusion matrix i analizu pogrešno klasificiranih tekstova), kao i sustavnije podešavanje hiperparametara neuronskog modela. Dodatno, zanimljiv smjer daljnjeg rada bila bi usporedba baseline i neuronskog modela na većem i uravnoteženom skupu podataka kako bi se jasnije procijenile prednosti sekvencijalnih modela u ovom tipu zadatka.

Literatura

- [1] D. Jurafsky, J. H. Martin, *Speech and Language Processing*.
- [2] PyTorch Documentation, <https://pytorch.org/docs/stable/index.html>
- [3] scikit-learn Documentation, <https://scikit-learn.org/stable/>